

Модульный и компонентный подход к описанию архитектуру ПС

Архитектура программного
средства

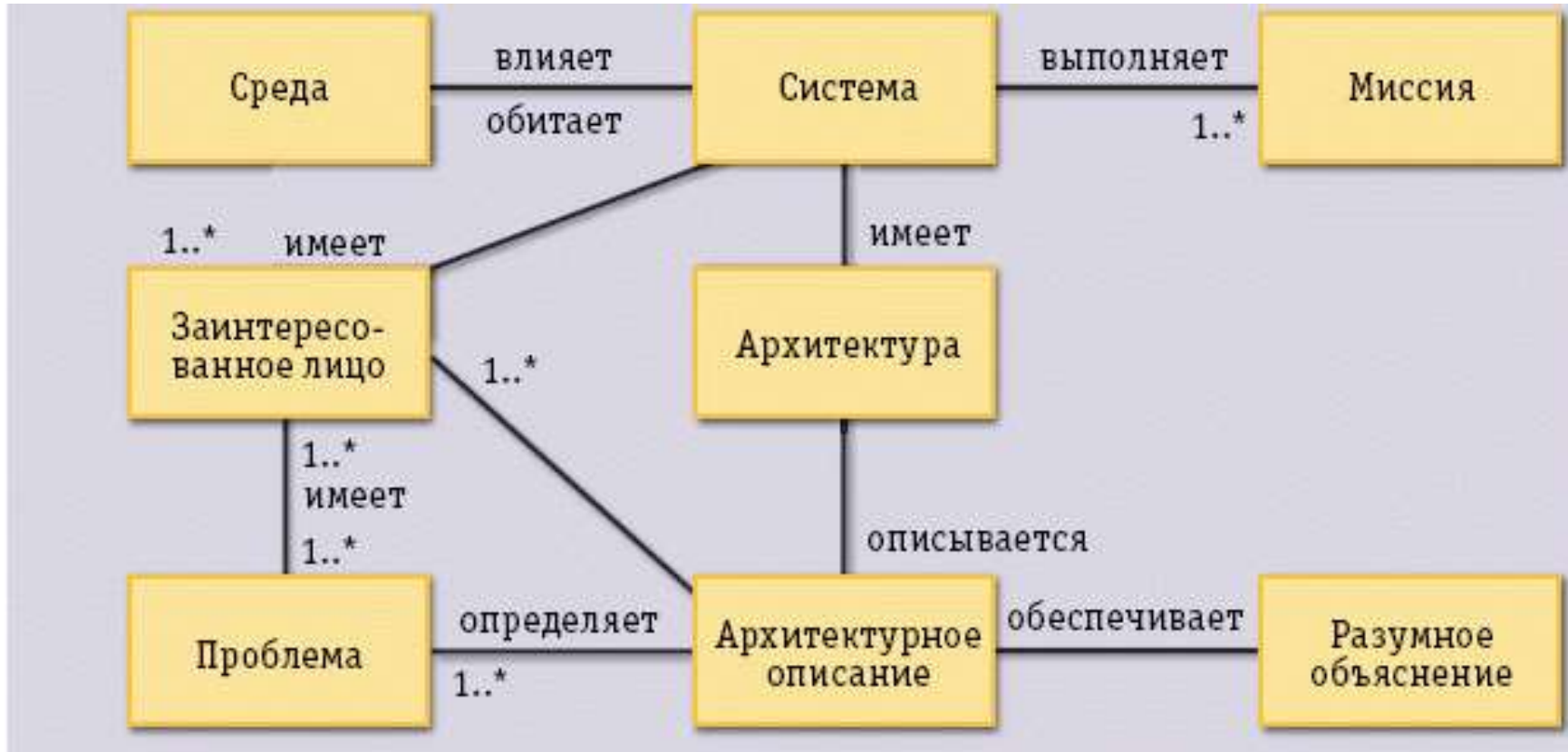
План

- Принципы модульного проектирования
- Понятие компонентного подхода к архитектуре
- Многоуровневые архитектуры
- MVC и его разновидности

Архитектура - это базовая организация системы, воплощенная в ее компонентах, их отношениях между собой и с окружением, а также принципы, определяющие проектирование и развитие системы.

[IEEE 1471]

Концептуальная модель

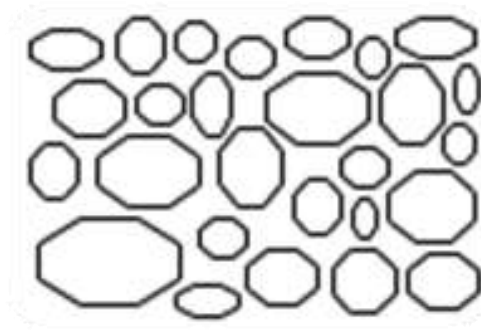


Архитектурный стиль

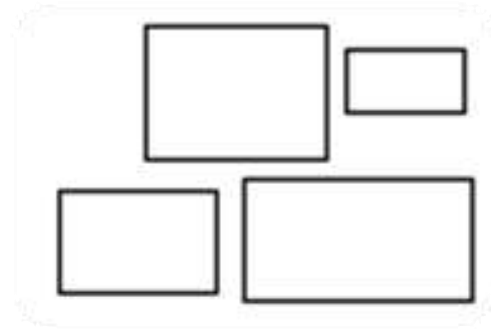
Монолитные системы – программные модули (объекты или функции) - компоненты



The Whole System



The Code Units
of the System

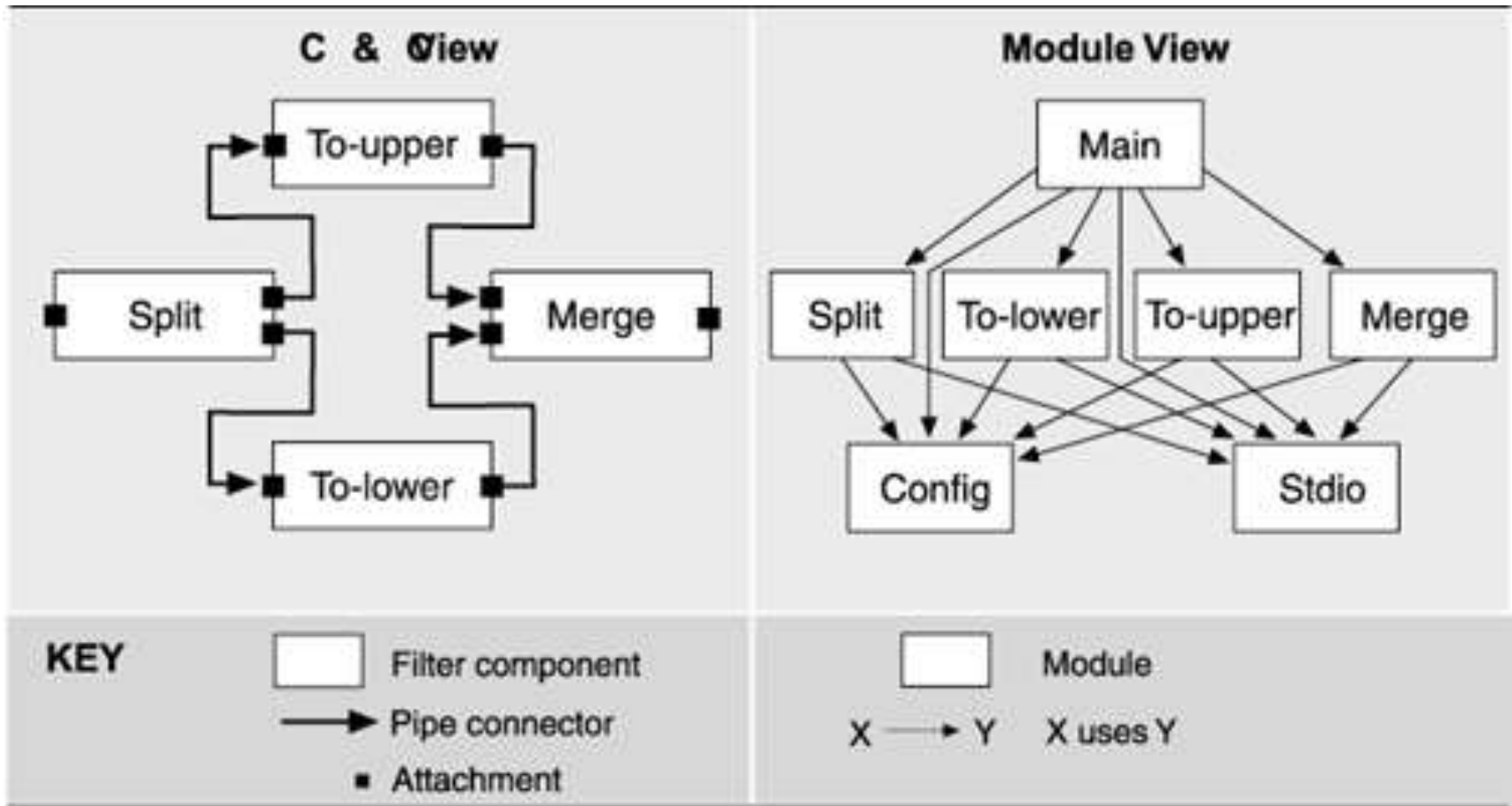


The Architectural
Components
of the System

Архитектура – это результат деятельности по проектированию

- **D-дизайн** (D-design, decomposition design) – декомпозиция структуры программного обеспечения в виде набора фрагментов или компонент;

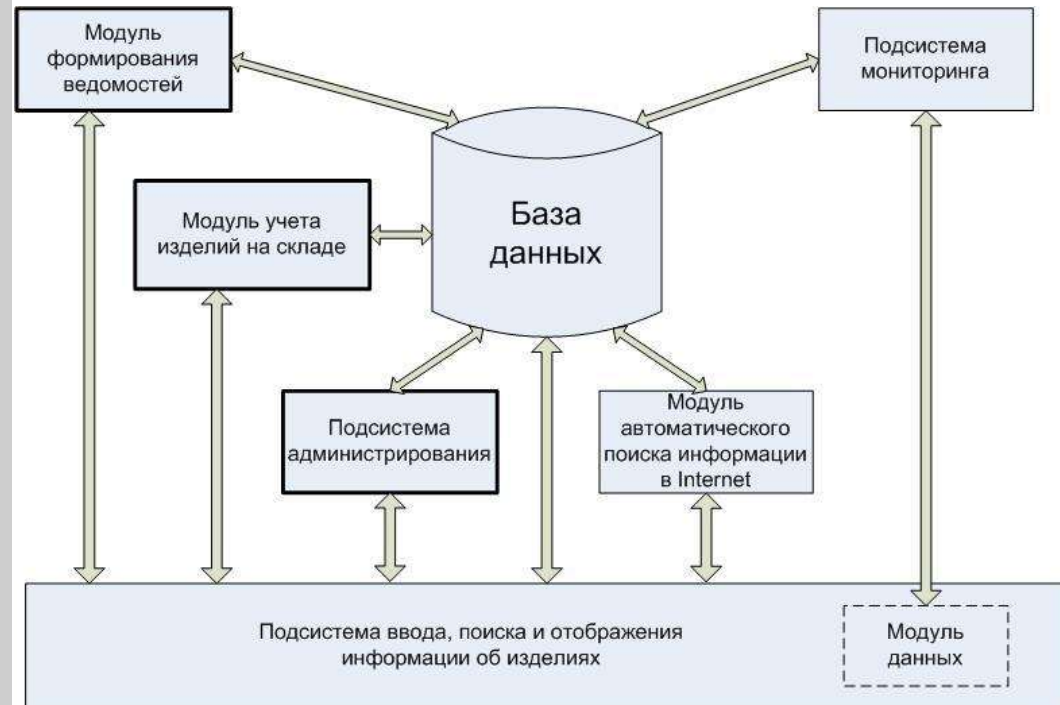
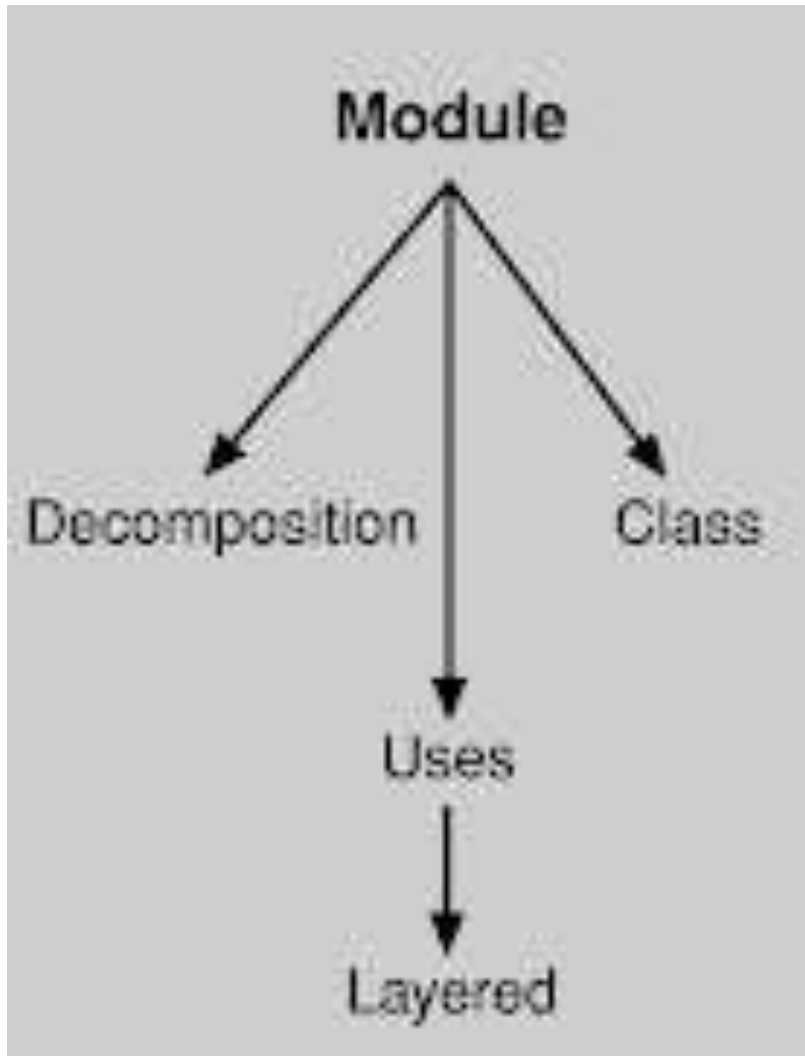
Модуль vs Компонент



Реализован физически, развертывание

Функциональность, часть программы

Архитектурное представление



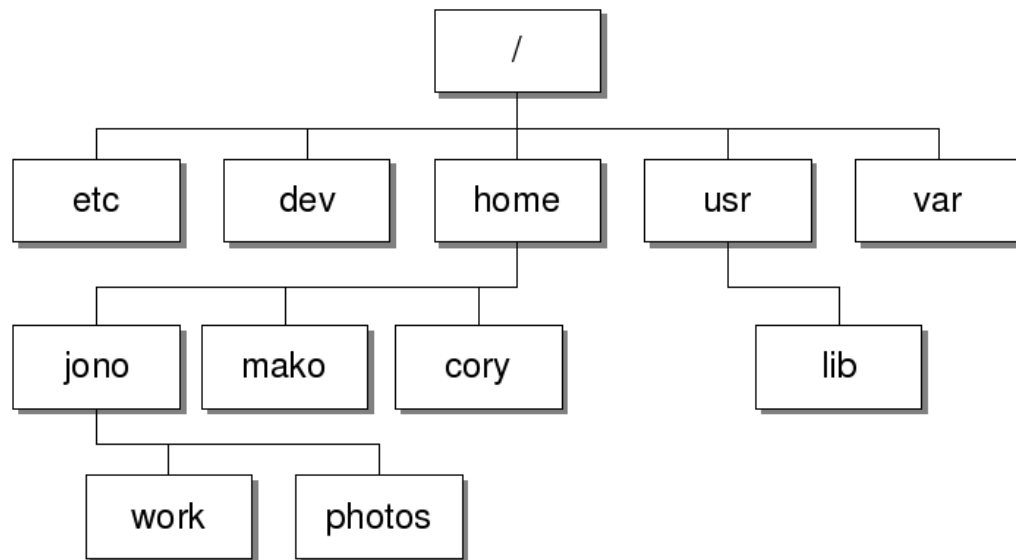
Модульное программирование

*Модульность в программировании
подобна честности в политике:
каждый утверждает, что она - одно
из его достоинств, но кажется,
никто не знает, что она собой
представляет ...*

Йодан Э.

Модульное программирование

— программирования, при котором вся программа разбивается на группу компонентов, называемых модулями, причем каждый из них имеет свой контролируемый размер, четкое назначение и детально проработанный интерфейс с внешней средой.



Основные концепции:

- В основе модульного программирования лежат три основные концепции:
 - *Принцип утаивания информации Парнаса*
При разработке программ формируется список проектных решений, которые особенно трудно понять или которые , скорее всего, будут меняться. Затем определяются отдельные модули, каждый из которых реализует одно из указанных решений. Большие программы должны использовать модули без каких-либо предварительных знаний об их внутренней структуре. Примерами удачных модулей могут служить программы ППП (пакетов прикладных программ) и стандартные процедуры.
 - *Сборочное программирование Цейтина.*
Модули – это программные «кирпичи», из которых строится программа.

Основные концепции:

- *Аксиома модульности* Коуэна
Модуль – независимая программная единица, служащая для выполнения некоторой определенной функции программы и для связи с остальной частью программы. Программная единица должна удовлетворять следующим условиям:
 - Блочность организации
 - Синтаксическая обособленность
 - Семантическая независимость
 - Общность данных
 - Полнота определения

Модуль характеризует:

- функциональная
завершенность;
- логическая
независимость;
- слабые
информационные связи
с другими
программными
модулями.

Имя модуля	
	содержит описание входных
спецификация	параметров, включенных модулей, изменяемых внешних переменных и выходных результатов
тело	инкапсулирует процесс обработки данных

Характеристики модуля (Майерс, 1980)

- Размер модуля
- Связность (прочность) модуля
- Сцепление модуля с другими модулями
- Рутинность (независимость от предыдущих обращений) модуля

Размер модуля

- Модуль не должен превышать 60 строк



```
if (uncoPaths.length > 0) do {
  for (i=0; i < uncoPaths.length; i++) do {
    if (uncoPaths[i].typeName == "group_item") do
      ProcessGroup(uncoPaths[i], untouchColor);
    else if (uncoPaths[i].typeName == "complex_item") do
      ProcessComplexItem(uncoPaths[i], untouchColor);
    else do
      ProcessItem(uncoPaths[i], untouchColor);
  }
}

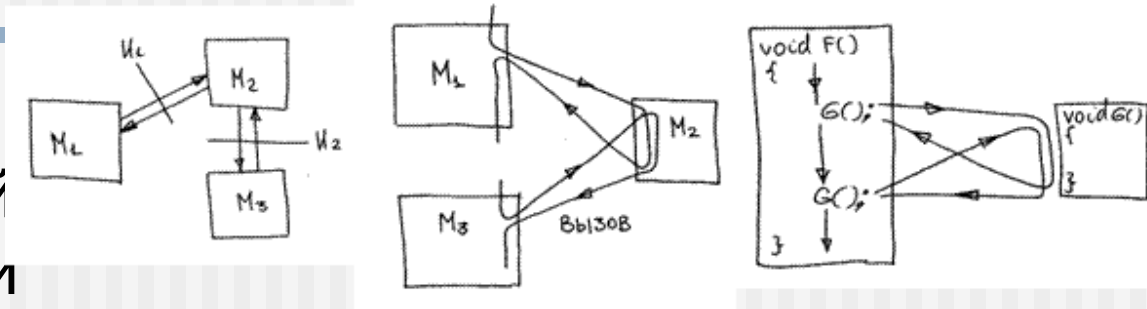
function ProcessItem(ob, col){
  if (ob.filled){
    if ( (ob.fillcolor.red <> col.red) or (ob.fillcolor.green <> col.green) or
      (ob.fillcolor.blue <> col.blvue) ) do
      ob.fillcolor = noColor;
    var selectionLen = curDoc.selection.length;
    for (i=0; i < selectionLen; i++) do {
      if (curDoc.selection[i].filled) do {
        if ( (curDoc.selection[i].fillcolor.red == untouchColor.red) and
          (curDoc.selection[i].fillcolor.green == untouchColor.green) and
          (curDoc.selection[i].fillcolor.blue == untouchColor.blue) ) do {
          throw(curDoc.selection[i].typename+
            " Incorrect object color.");
        }
      }
    }
  }
}
```

Сцепление модулей

- мера относительной независимости модулей от других модулей.

Независимые модули могут быть модифицированы без переделки других модулей.

Чем слабее сцепление модуля, тем лучше.



Coupling

- способ и степень взаимозависимости между программными модулями;
- сила взаимосвязей между модулями;
- мера того, насколько взаимозависимы разные подпрограммы или модули.



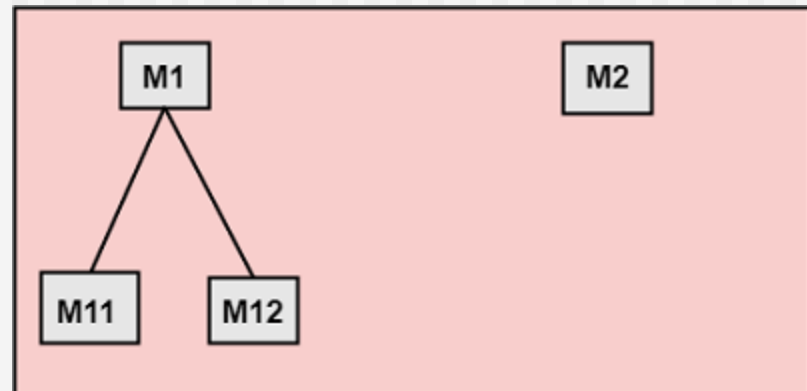
Существуют различные методы уменьшения зацепления (англ. decoupling). Как правило, они описаны в виде шаблонов проектирования.

Типы сцепления

Независимые модули – идеальный случай.

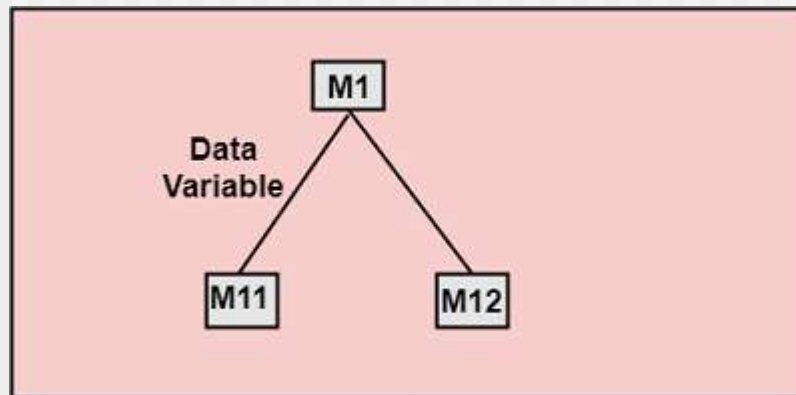
В этом случае модули ничего не знают друг о друге.

Взаимодействие модулей организуется через их интерфейсы, когда выходные данные одного модуля передаются на вход другого.

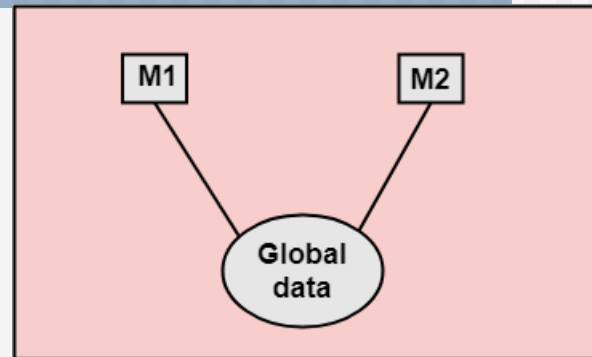


Типы сцепления

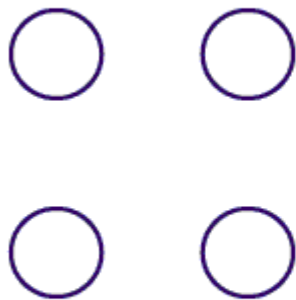
- *Сцепление по данным (параметрическое)*
– это сцепление, когда данные передаются модулю как значения его параметров или как результат его обращения к другому модулю для вычисления некоторой функции



Общее сцепление

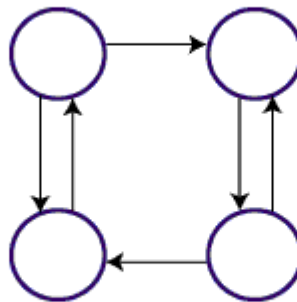


Module Coupling



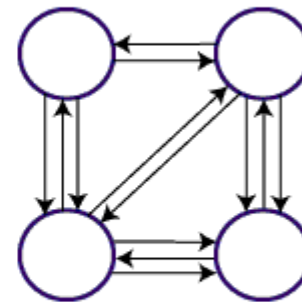
Uncoupled: no dependencies

(a)



Loosely Coupled: Some dependencies

(b)



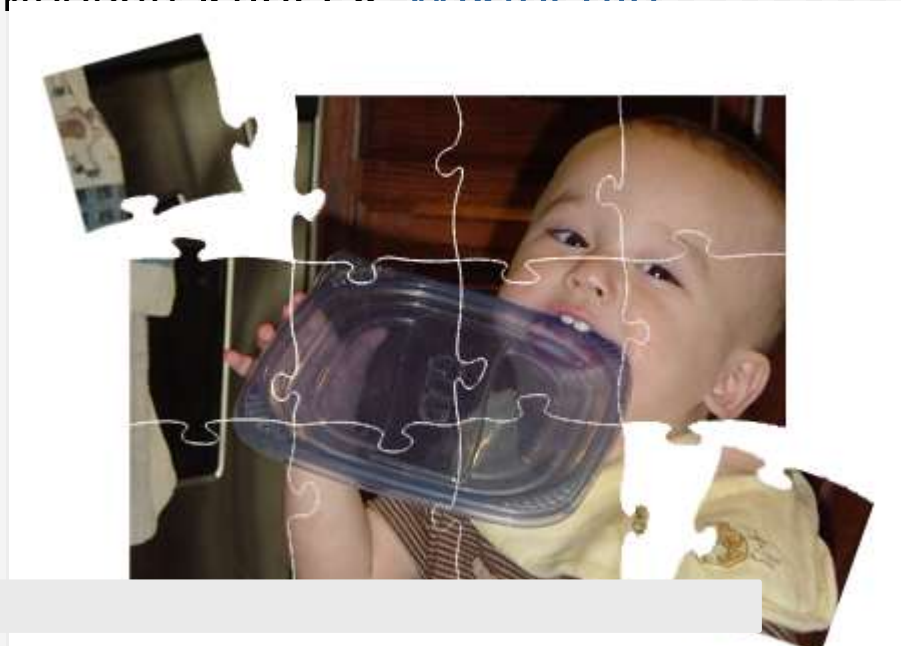
Highly Coupled: Many dependencies

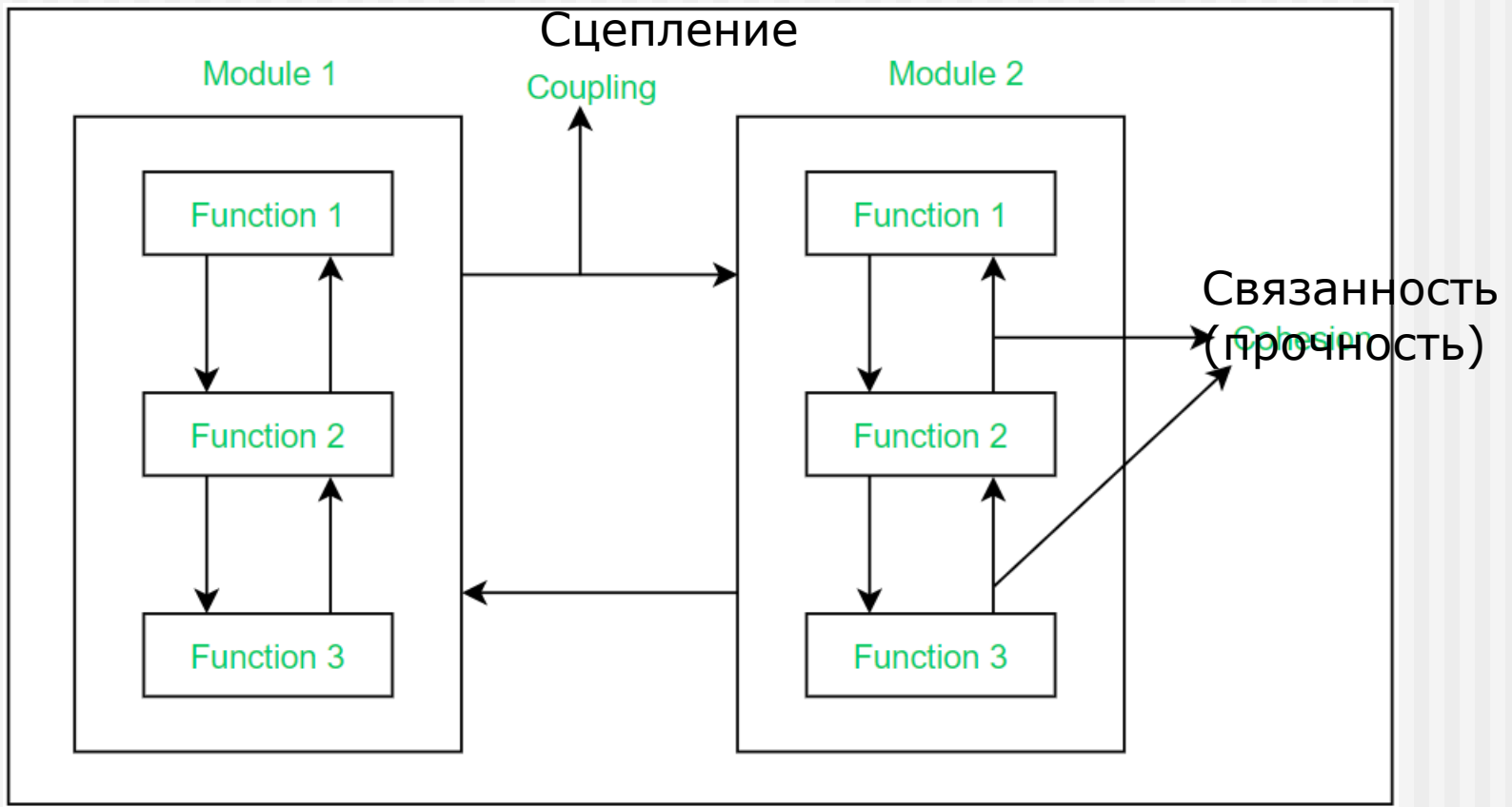
(c)

COHESION

- мера силы взаимосвязанности элементов внутри модуля;
- способ и степень, в которой задачи, выполняемые некоторым программным модулем, связаны друг с другом.

Макконнелл объясняет понятие связности на примере класса (как частного случая модуля): «связность характеризует то, насколько хорошо все методы класса или все фрагменты метода соответствуют главной цели, — иначе говоря, насколько сфокусирован класс» [Wikipedia](#)

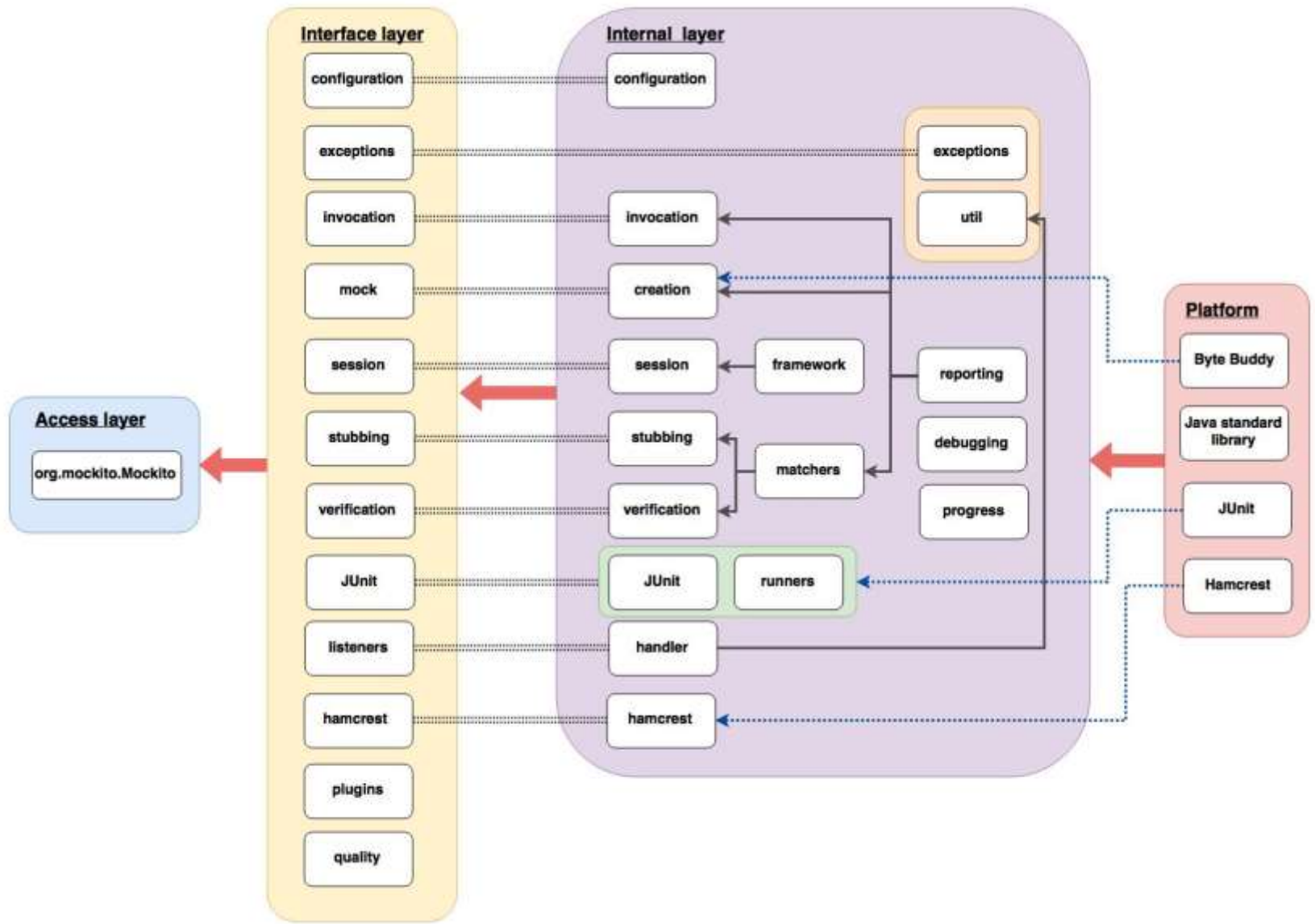




Рутинность модуля

- *Рутинность модуля* – это независимость модуля от предыдущих обращений к нему.

Mockito - фреймворк для имитации модульного тестирования на Java



Преимущества модульного программирования:

- ***Функциональные компоненты модульной программы могут быть написаны и отлажены порознь***
- ***Модульную программу проще проектировать, легче сопровождать и модифицировать***

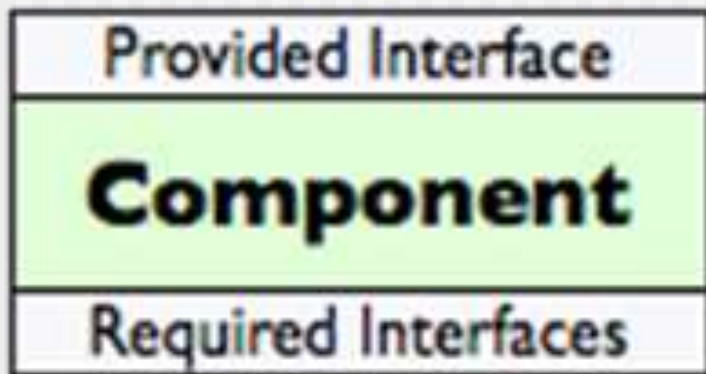
Стандартные модули

- 1) стандартные модули экономят время программирования;
- 2) они также могут экономить память компьютера и выполняться максимально быстро;
- 3) использование стандартных модулей защищает от ошибок программирования.

Разработка и использование стандартных библиотечных программ является одним из путей построения модульного программирования

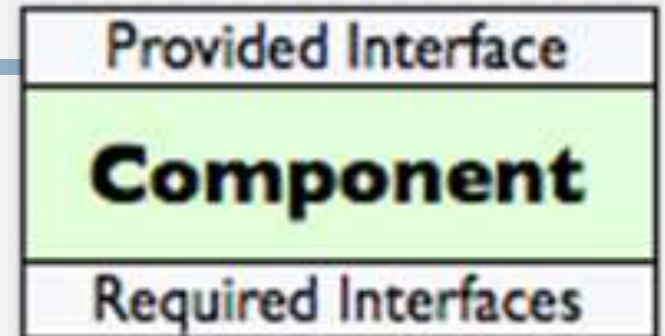
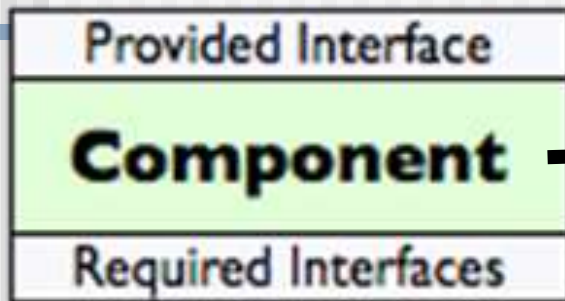
КОМПОНЕНТНЫЙ

Компонент

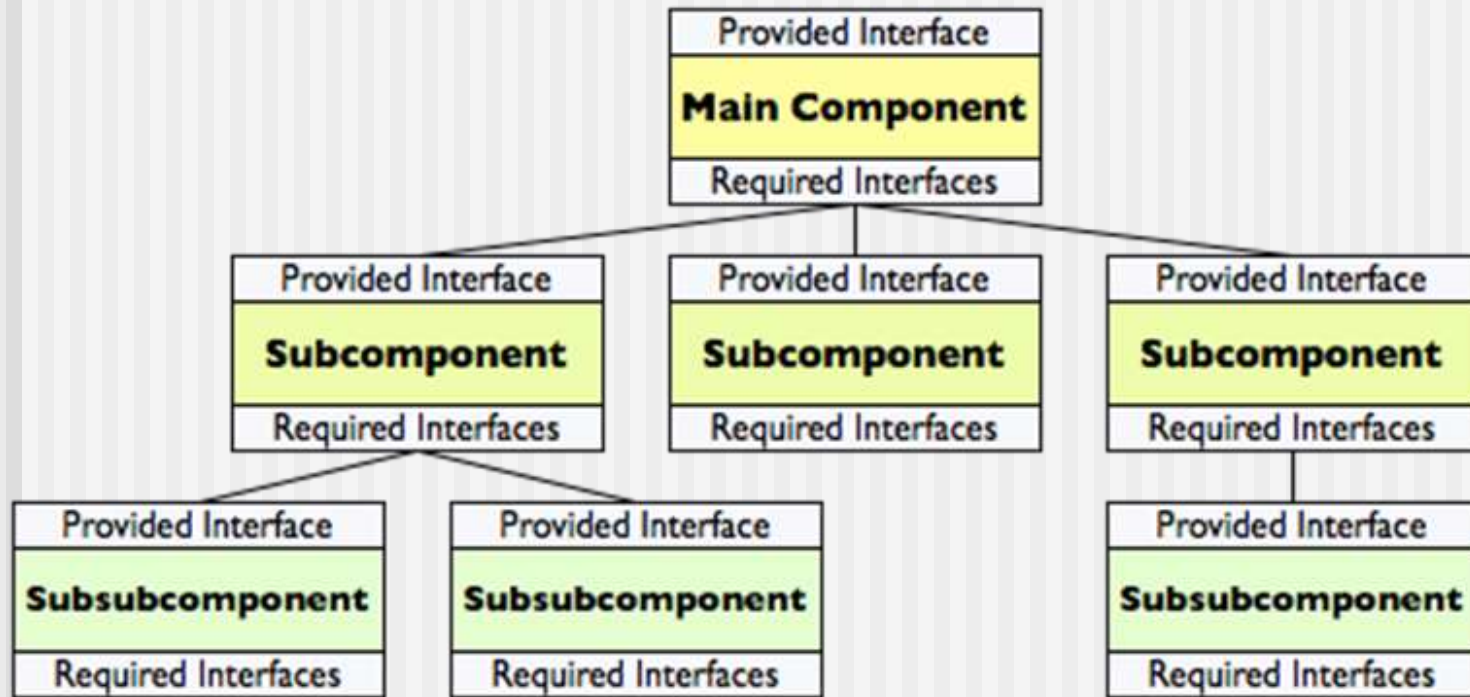


- Интерфейсы, к-е он предоставляет
- Компонент
- Интерфейсы, к-е ему необходимы

Соединение



Иерархическая архитектура

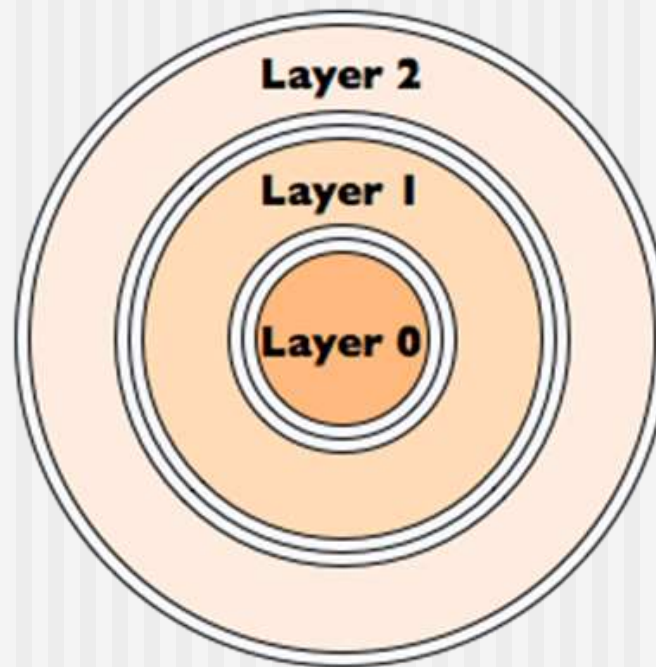
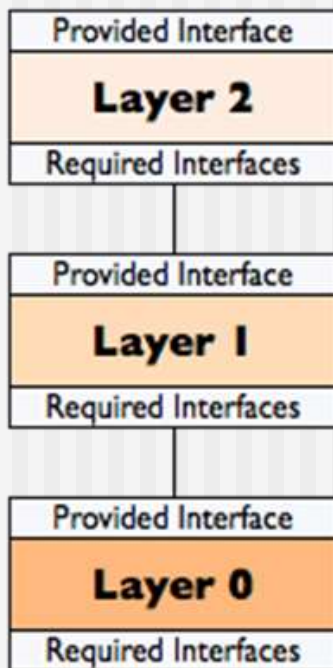


структура

Архитектура «Каналы и фильтры»



Многослойная архитектура



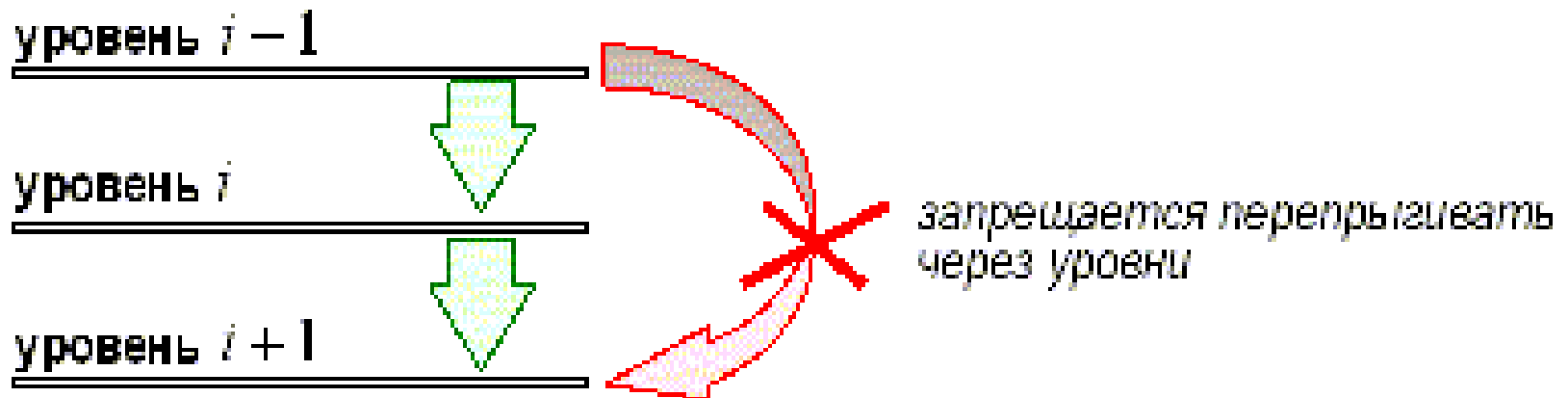
структура

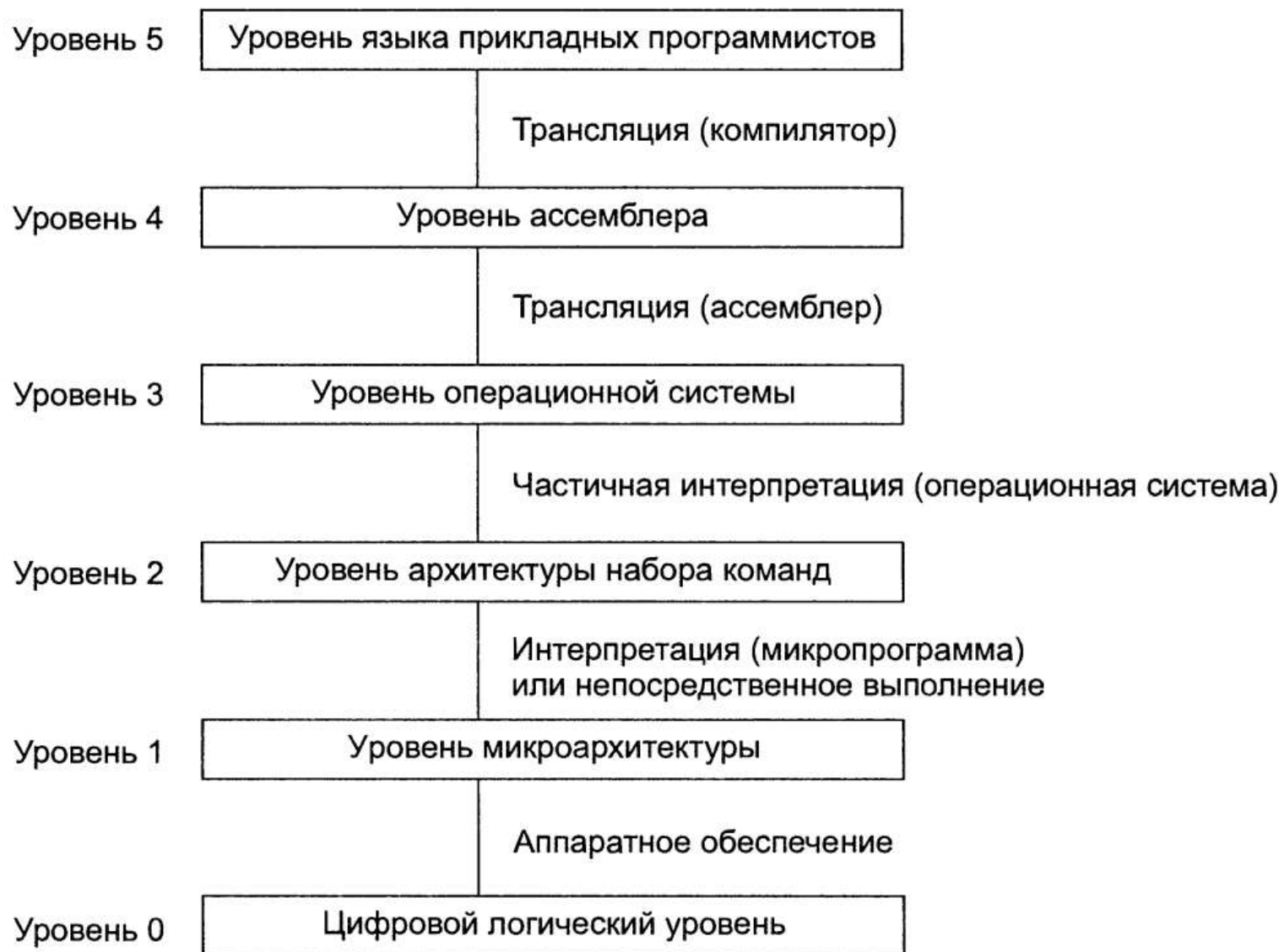
Многоуровневая архитектура



свойство как разделение ответственности

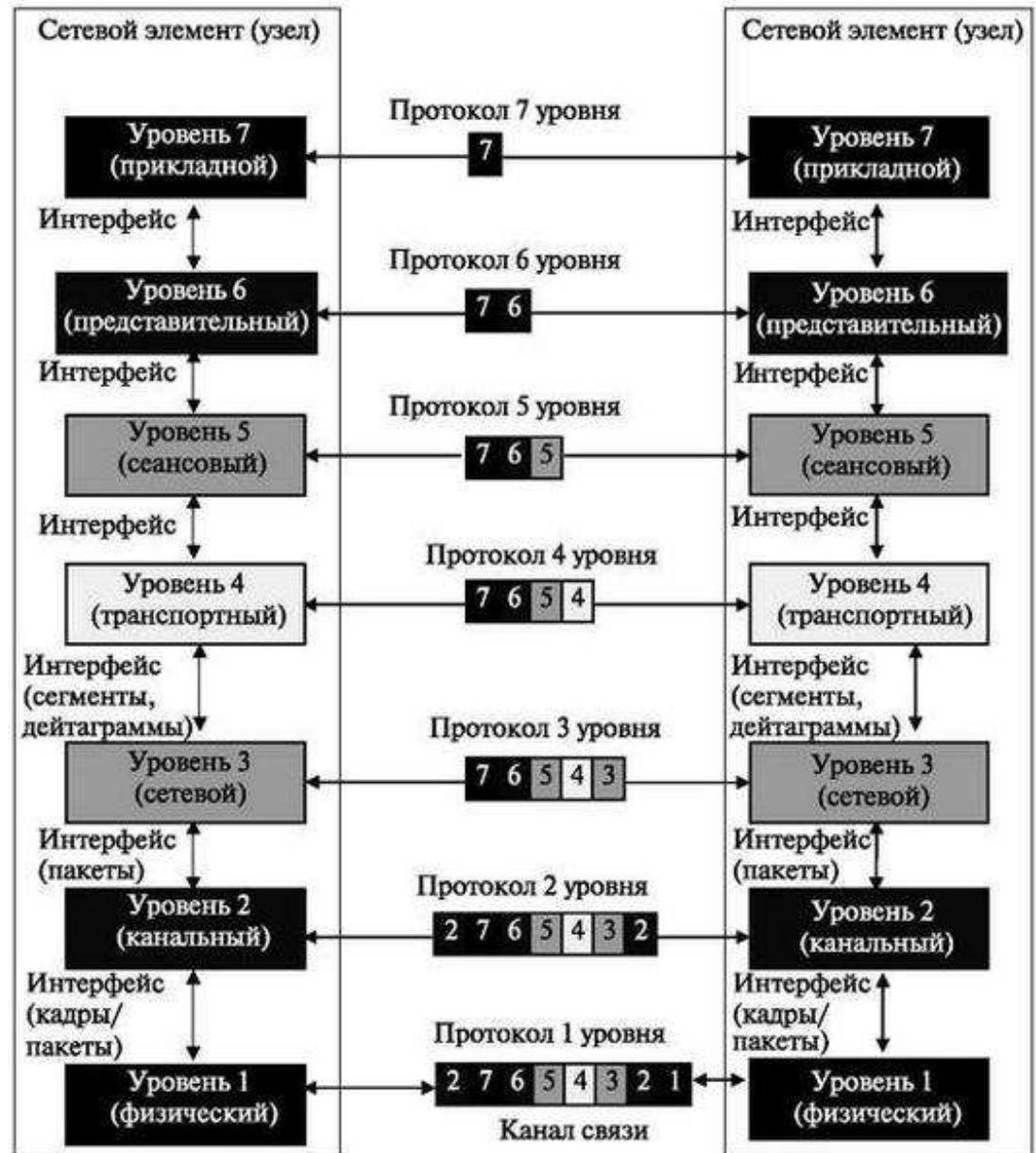
- через интерфейс





Шестиуровневый компьютер. Способ поддержки каждого уровня указан под ним, в скобках дано название соответствующего программного обеспечения

модель взаимодействия открытых систем (Open System Interconnection, OSI)



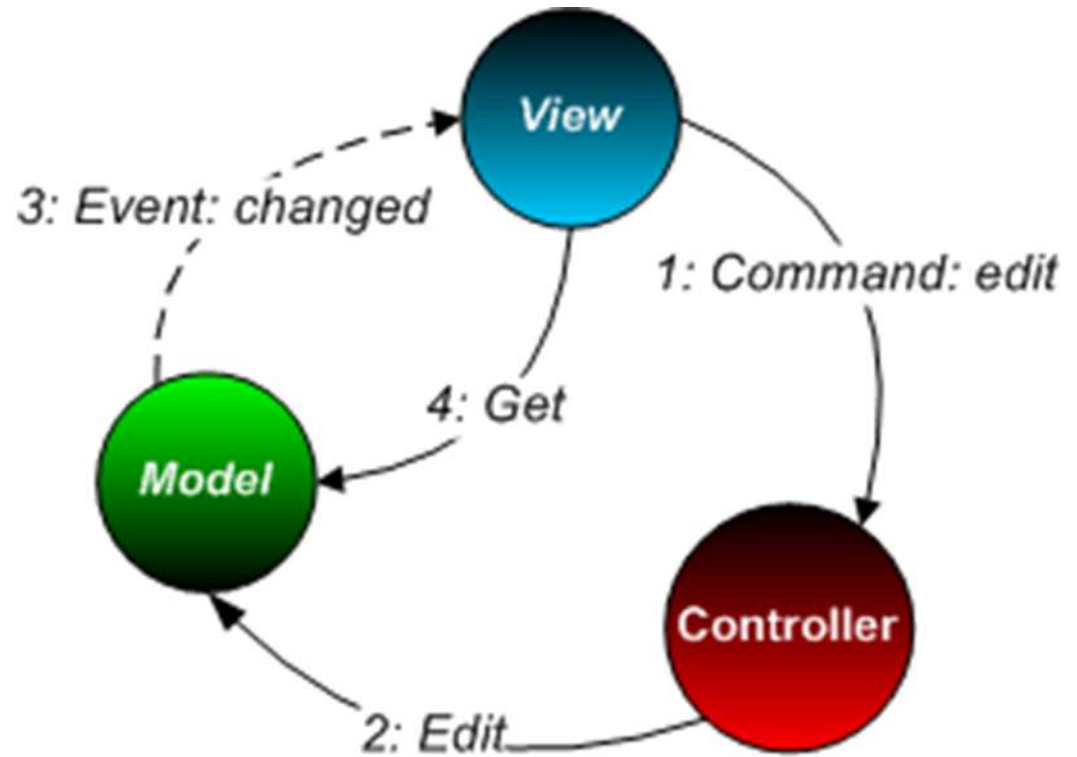
Запись в файл

Имя функции	Уровень	Назначение
↓ <code>fwrite</code>	Библиотека <code>stdio</code>	Буферизация данных
↓ <code>write</code>	Системный вызов	Интерфейс POSIX
Граница между пространствами пользователя и ядра		
↓ <code>sys_write</code>	Точка входа в ядро	Обработка аргумента
↓ <code>vn_write</code>	<code>Vnode</code>	Адаптация к интерфейсу <code>vnode</code>
↓ <code>VOP_WRITE</code>	Коммутатор <code>Vnode</code>	Система интерфейсов
↓ <code>ext2fs_write</code>	Файловая система	Структура данных
↓ <code>bwrite</code>	Кэширование в ядре	Буферизация на уровне ядра
↓ <code>VOP_STRATEGY</code>	Коммутатор <code>Vnode</code>	Система интерфейсов
↓ <code>ufs_strategy</code>	Блок	Логические блоки
↓ <code>VOP_STRATEGY</code>	Коммутатор <code>Vnode</code>	Система интерфейсов
↓ <code>spec_strategy</code>	Коммутатор устройств	Система устройств
↓ <code>sdstrategy</code>	Асинхронные операции	Очереди ввода-вывода
Граница между синхронным и асинхронным кодом		
↑ <code>scsi_pi_done</code>	Интерфейс SCSI	Постановка в очередь, извещение
↑ <code>aha_done</code>	Команды устройства	Обработка результата
↑ <code>aha_finish_ccbs</code>	Управляющие блоки	Обработка результата
↑ <code>aha_intr</code>	Прерывание устройства	Обработка ящика сообщений
↑ <code>INTR</code>	Системное прерывание	Обработка прерываний

Разделение по функциональности



- 1. функции ввода и отображения данных (обеспечивают взаимодействие с пользователем);
- 2. прикладные функции, характерные для данной предметной области;
- 3. функции управления ресурсами (файловой системой, базой данных и т.д.)



СТИЛЬ MVC

Модель

- Содержит данные предметной области;
- Содержит логику для создания, управления и модификации данных предметной области;
- Предоставляет интерфейс для доступа к данным модели и операциям с ним

Не должна:

- Показывать Детали
- Трансформации пользователем
- Логика отображения

Контроллер – «соединительная ткань»

- Содержит действия, требующиеся для обновления модели на основе взаимодействия с пользователем

Не должен
содержать логику
управления:

- внешним видом данных;
- Постоянством данных

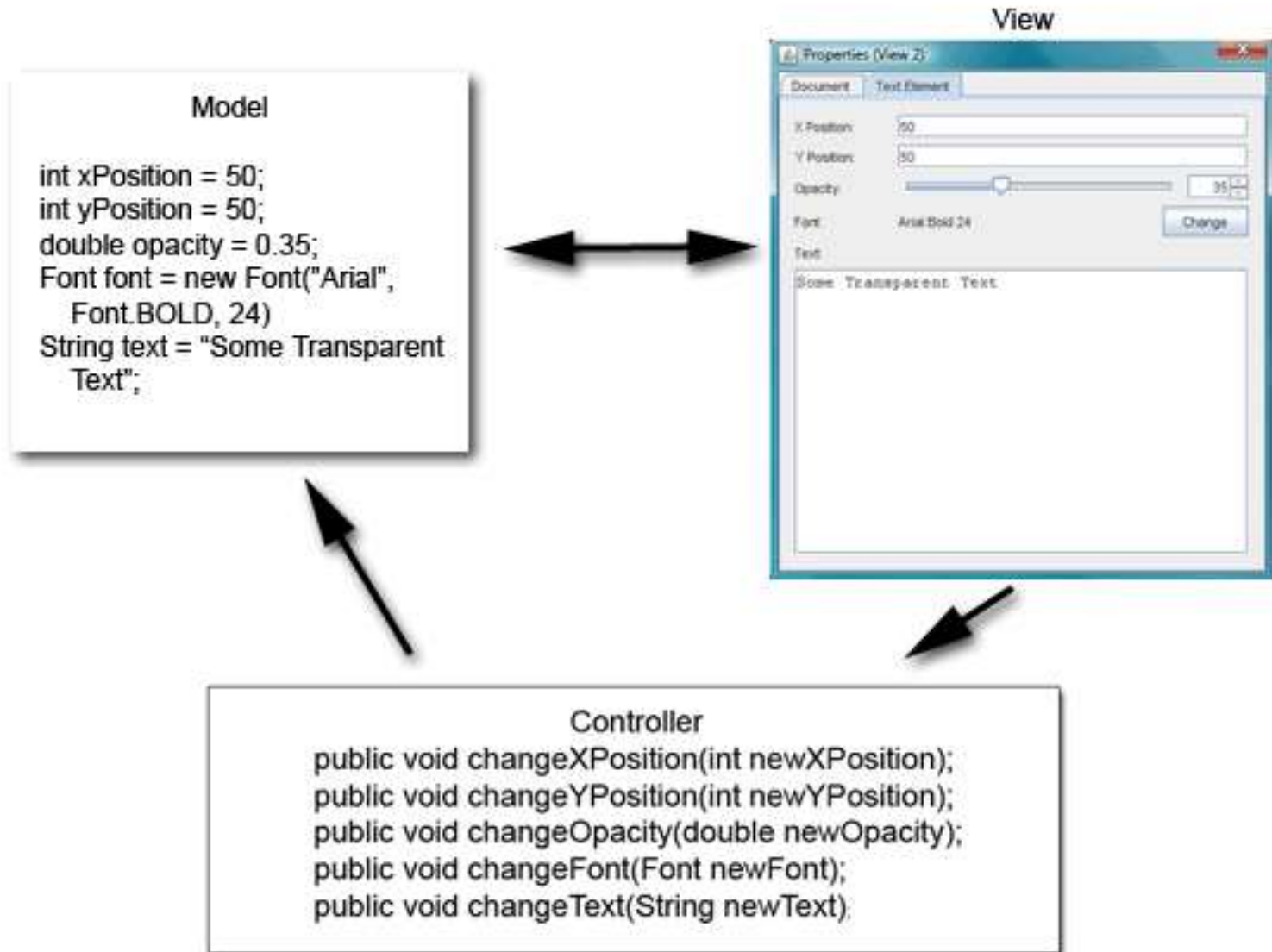
Представление

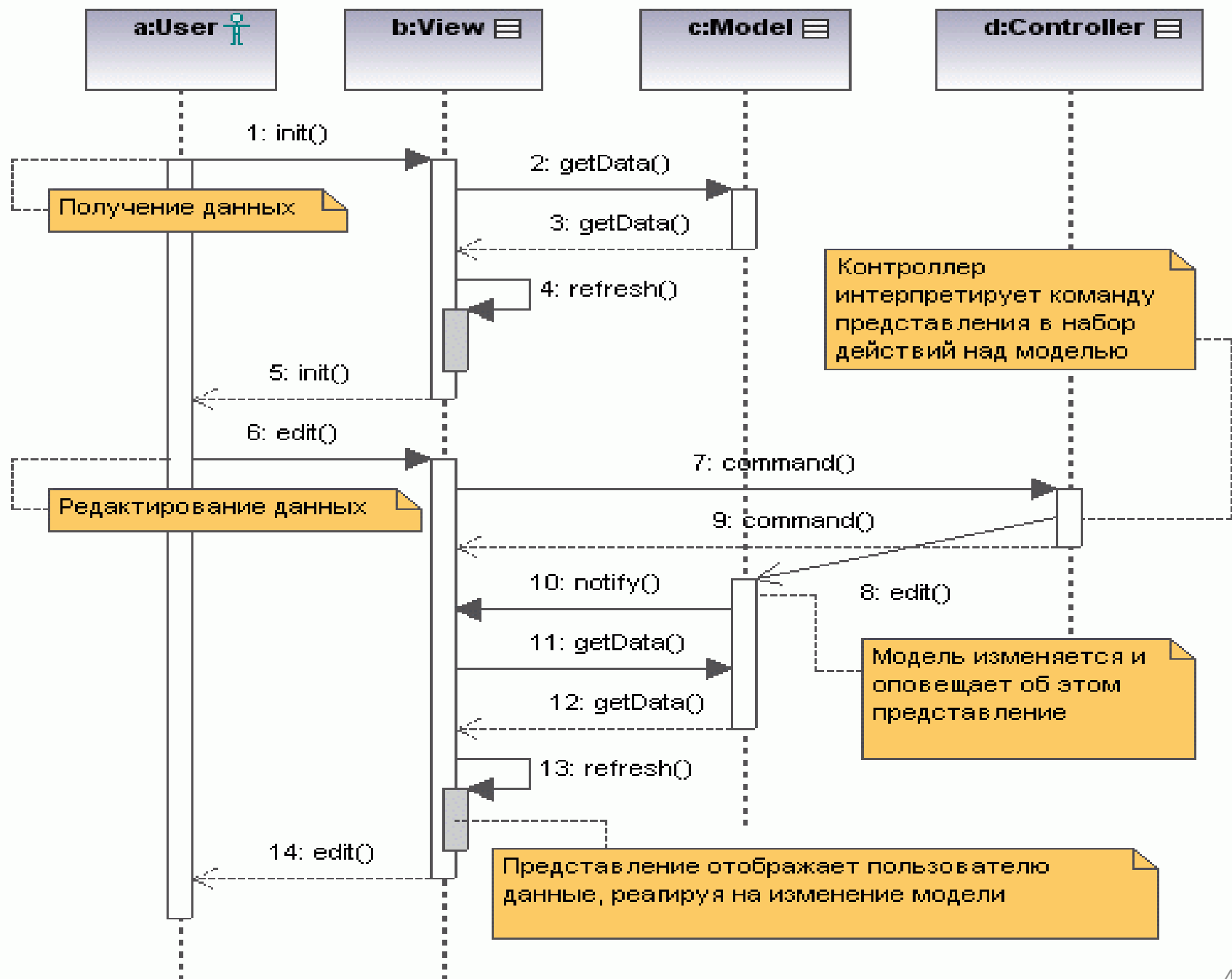
- Содержит логику и разметку, необходимые для показа данных пользователю

Нет

- сложной логики
- Манипуляции с моделью

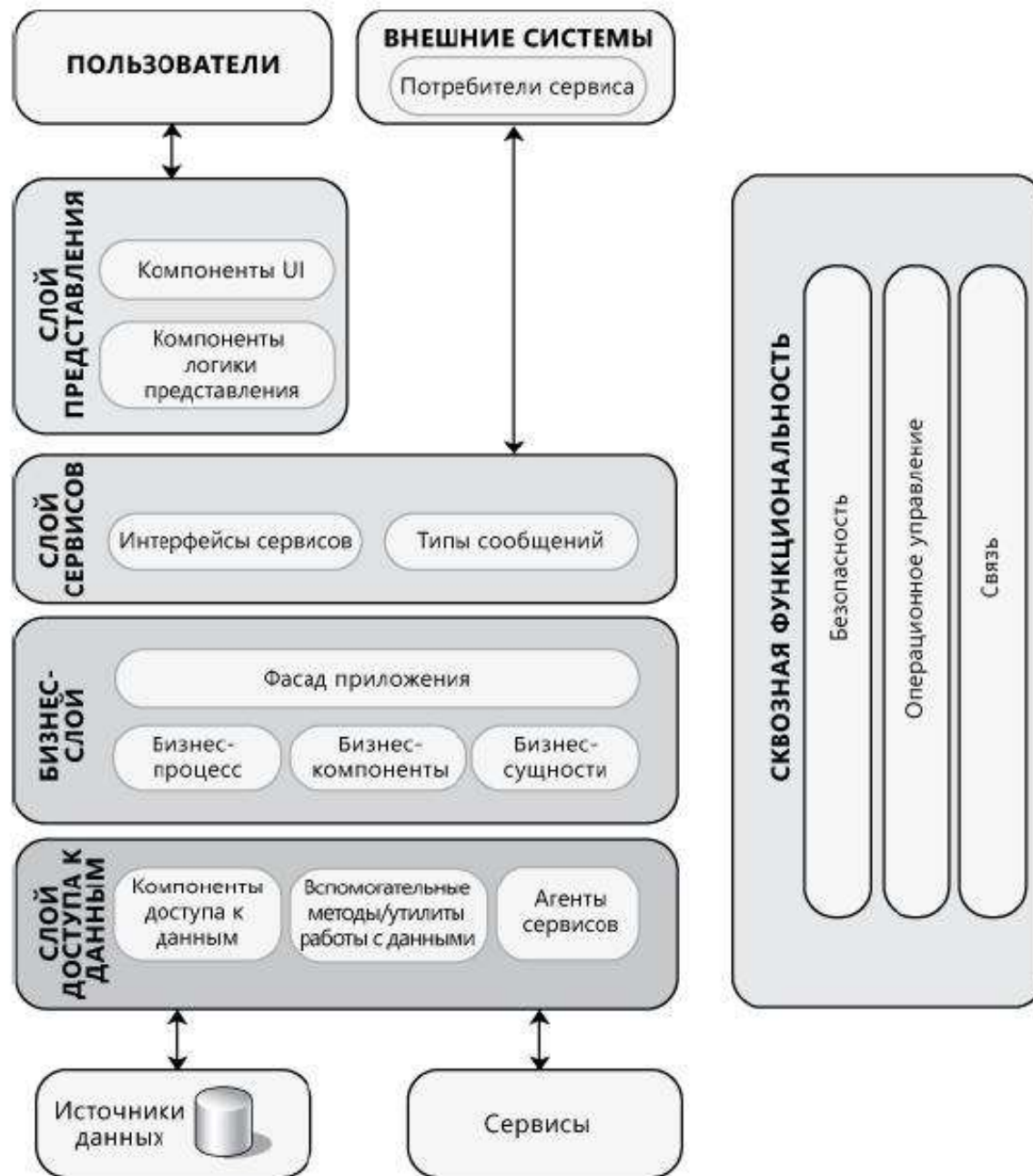
Java SE

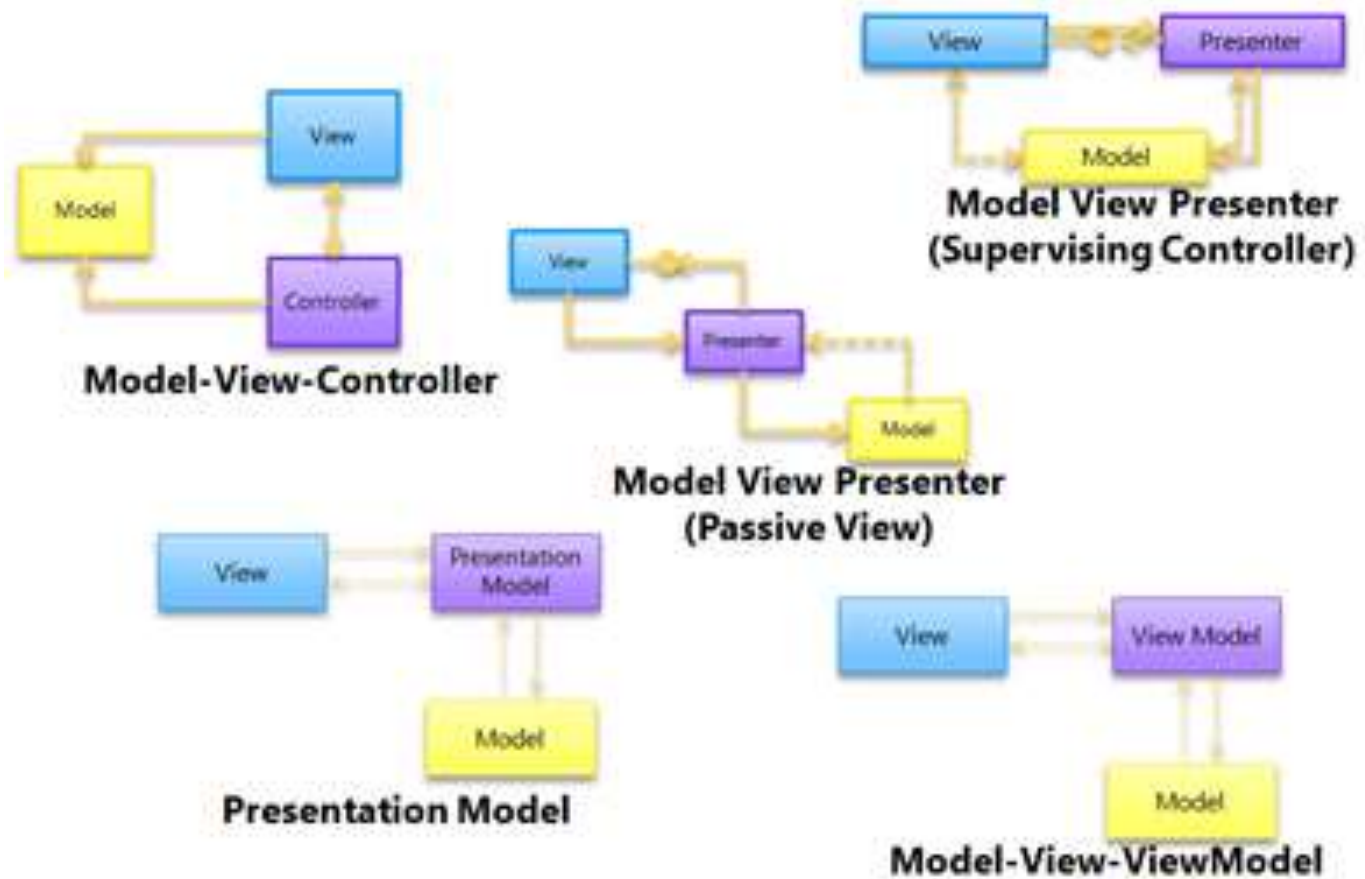




MS

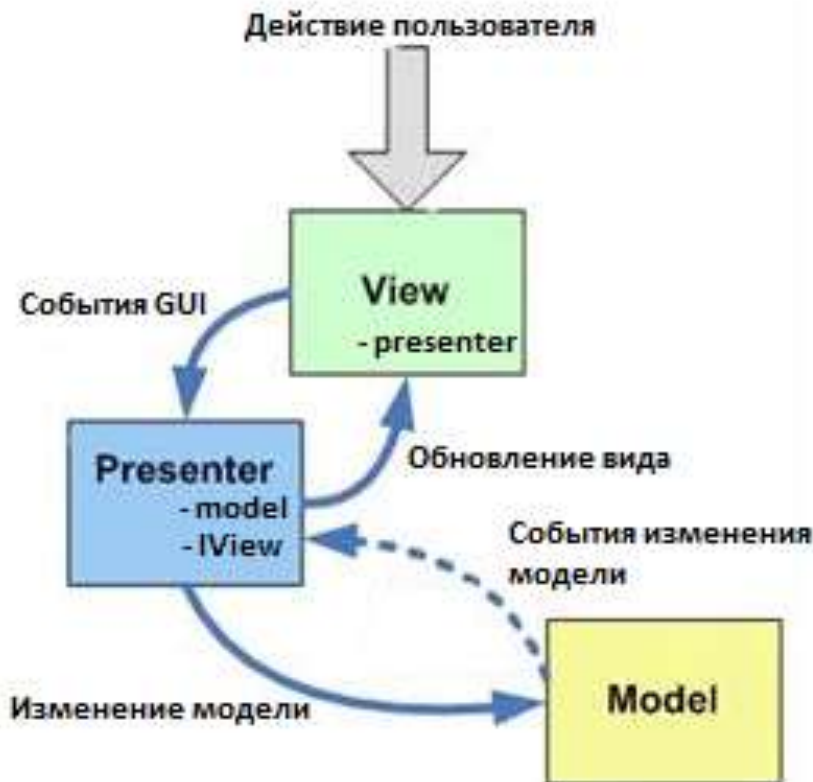
На
функциональные
слои





РАЗНОВИДНОСТИ MVC

Model-View-Presenter



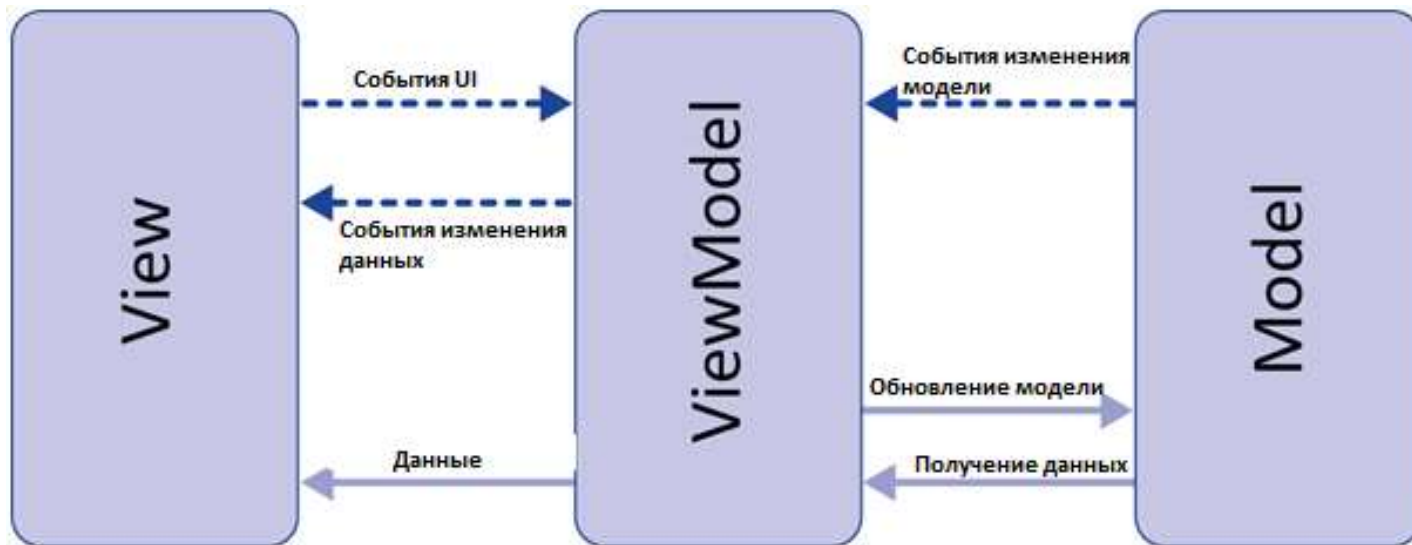
Model-View-Presenter

Двухсторонняя коммуникация с представлением;

- Представление взаимодействует напрямую с презентером, путем вызова соответствующих функций или событий экземпляра презентера;
- Презентер взаимодействует с View путем использования специального интерфейса, реализованного представлением;
- *Один экземпляр презентера связан с одним отображением.*

Пример использования: Windows Forms

Model-View-View Model



- Двухсторонняя коммуникация с представлением;
- View-модель — это абстракция представления.
- View-модель не имеет ссылки на интерфейс представления (IView). Изменение состояния View-модели автоматически изменяет представление и наоборот, поскольку используется механизм связывания данных (**Bindings**)
- Один экземпляр View-модели связан с одним отображением.

